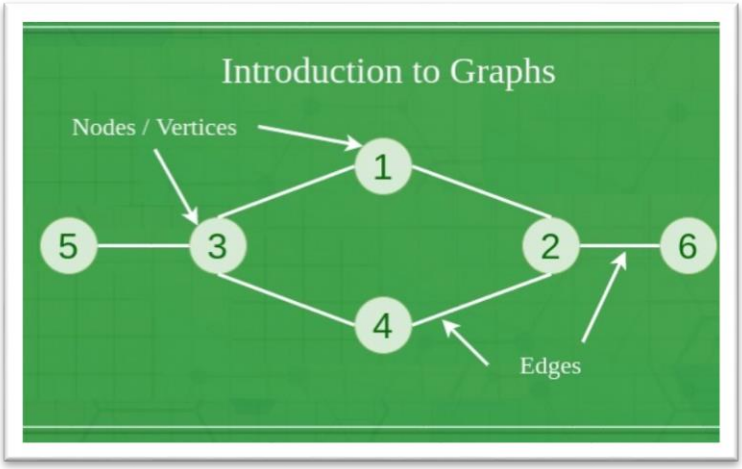
GRAPHS

What is Graph Data Structure?

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by G(E, V).



Components of a Graph

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.
- **Edges:** Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labeled/unlabelled.

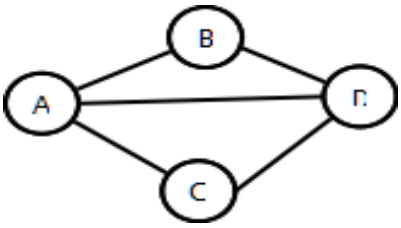
Graphs are used to solve many real-life problems. Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network. Graphs are also used in social networks like linkedIn, Facebook. For example, in Facebook, each person is represented with a vertex(or node). Each node is a structure and contains information like person id, name, gender, locale etc

- Definitions: Vertices, edges, paths, etc
- Representations: Adjacency list and adjacency matrix

Definitions: Graph, Vertices, Edges

- Define a graph G = (V, E) by defining a pair of sets:
  1. V = a set of **vertices**
  2. E = a set of **edges**

- Edges:
    1. Each edge is defined by a pair of vertices
    2. An edge **connects** the vertices that define it
    3. In some cases, the vertices can be the same
- Vertices:
    1. Vertices also called **nodes**
    2. Denote vertices with labels
- Representation:
    1. Represent vertices with circles, perhaps containing a label
    2. Represent edges with lines between circles
- Example:
    1. V = {A,B,C,D}
    2. E = {(A,B),(A,C),(A,D),(B,D),(C,D)}



Motivation

- Many algorithms use a graph representation to represent data or the problem to be solved
- Examples:
    - Cities with distances between
    - Roads with distances between intersection points
    - Course prerequisites
    - Network
    - Social networks
    - Program call graph and variable dependency graph

Graph Classifications

- There are seveal common kinds of graphs
    - Weighted or unweighted
    - Directed or undirected
    - Cyclic or acyclic

- Choose the kind required for problem and determined by data

Kinds of Graphs: Weighted and Unweighted

- Graphs can be classified by whether or not their edges have **weights**

- **Weighted graph**: edges have a weight
  - Weight typically shows cost of traversing
  - Example: weights are distances between cities

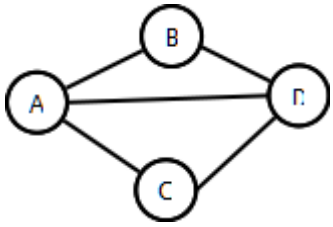
- **Unweighted graph**: edges have no weight
  - Edges simply show connections
  - Example: course prereqs
  -

Kinds of Graphs: Directed and Undirected

- Graphs can be classified by whether or their edges are have direction
  - **Undirected Graphs**: each edge can be traversed in **either direction**
  - **Directed Graphs**: each edge can be traversed **only in a specified direction**

Undirected Graphs

- **Undirected Graph**: no implied direction on edge between nodes
  - The example from above is an undirected graph



  - In diagrams, edges have no direction (ie they are not arrows)

  - Can traverse edges in either directions

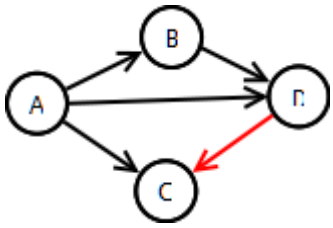
- In an undirected graph, an edge is an **unordered** pair
  - Actually, an edge is a set of 2 nodes, but for simplicity we write it with parens
    - For example, we write (A, B) instead of {A, B}
    - Thus, (A,B) = (B,A), etc
    - If (A,B) ∈ E then (B,A) ∈ E

  - Formally: ∀ u,v ∈ E, (u,v)=(v,u) and u ≠ v

- A node normally does not have an edge to itself

Directed Graphs

- **Digraph**: A graph whose edges are directed (ie have a direction)
  - Edge drawn as arrow
  - Edge can only be traversed in direction of arrow
  - Example: E = {(A,B), (A,C), (A,D), (B,C), (D,C)}



  - Examples: courses and prerequisites, program call graph

- In a digraph, an edge is an **ordered** pair
  - Thus: (u,v) and (v,u) are not the same edge
  - In the example, (D,C) ∈ E, (C,D) ∉ E
  - What would edge (B,A) look like? Remember (A,B) ≠ (B,A)

- A node can have an edge to itself (eg (A,A) is valid)

<u>Subgraph</u>

- If graph G=(V, E)
  - Then Graph G'=(V',E') is a **subgraph** of G if V' ⊆ V and E' ⊆ E

<u>Degree of a Node</u>

- The **degree** of a node is the number of edges the node is used to define

- Give the example for:
  - Degree 2: B and C
  - Degree 3: A and D
  - A and D have **odd degree**, and B and C have **even degree**

- Can also define **in-degree** and **out-degree**
  - In-degree: Number of edges pointing **to** a node
  - Out-degree: Number of edges pointing **from** a node

Graphs: Terminology Involving Paths

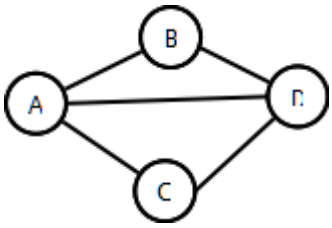
- **Path**: sequence of vertices in which each pair of successive vertices is connected by an edge

- **Cycle**: a path that starts and ends on the same vertex

- **Simple path**: a path that does not cross itself
    - That is, no vertex is repeated (except first and last)
    - Simple paths cannot contain cycles

- **Length** of a path: Number of edges in the path
    - Sometimes the sum of the weights of the edges

- Examples

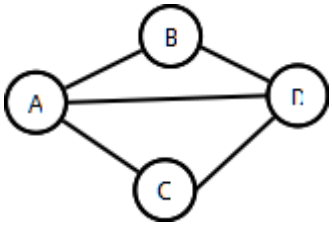


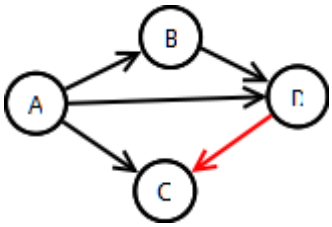
    - A sequence of vertices: (A, B, C, D) [Is this path, simple path, cycle?]
    - (A, B, D, A, C) [path, simple path, cycle?]
    - Cycle: ?
    - Simple Cycle: ?
    - Lengths?

Cyclic and Acyclic Graphs

- A **Cyclic** graph contains cycles
    - Example: roads (normally)

- An **acyclic** graph contains no cycles
    - Example: Course prereqs!
- Examples - Are these cyclic or acyclic?





Connected and Unconnected Graphs and Connected Components

- An undirected graph is **connected** if every pair of vertices has a path between it
  - Otherwise it is unconnected
  - Give an example of a connected graph

- An unconnected graph can be broken in to **connected components**

- A directed graph is **strongly connected** if every pair of vertices has a path between them, in **both directions**

Trees and Minimum Spanning Trees

- Tree: undirected, connected graph with no cycles
  - Example ...
  - If G=(V, E) is a tree, how many edges in G?
- Spanning tree: a **spanning tree** of G is a connected subgraph of G that is a tree
- **Minimum spanning tree** (MST): a spanning tree with minimum weight
- Spanning trees and minimum spanning tree are not necessarily unique

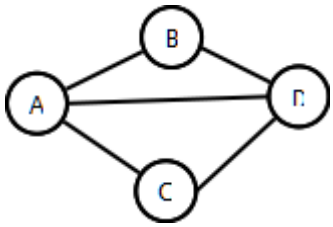
Data Structures for Representing Graphs

- Two common data structures for representing graphs:
  - Adjacency lists
  - Adjacency matrix

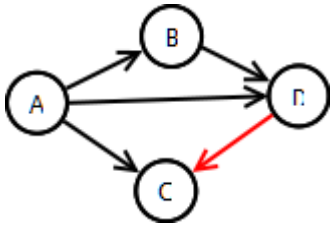
Adjacency List Representation

- Each node has a list of adjacent nodes

- Example (undirected graph):
  - A: B, C, D
  - B: A, D
  - C: A, D
  - D: A, B, C



- Example (directed graph):
  - A: B, C, D
  - B: D
  - C: Nil
  - D: C



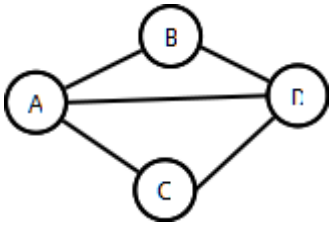
- Weighted graph can store weights in list
- Space: Θ(V + E) (ie |V| + |E|)
- Time:
  - To visit each node that is adjacent to node u: Θ(degree(u))
  - To determine if node u is adjacent to node v: Θ(degree(u))

Adjacency Matrix Representation

- **Adjacency Matrix**: 2D array containing weights on edges
  - Row for each vertex
  - Column for each vertex
  - Entries contain weight of edge from row vertex to column vertex
  - Entries contain ∞ (ie Integer'last) if no edge from row vertex to column vertex
  - Entries contain 0 on diagonal (if self edges not allowed)

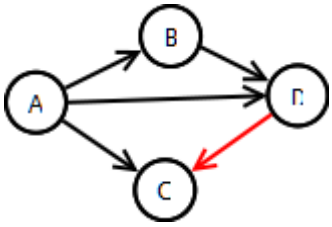
Example undirected graph (assume self-edges not allowed):

	A	B	C	D
A	0	1	1	1
B	1	0	∞	1
C	1	∞	0	1
D	1	1	1	0



Example directed graph (assume self-edges allowed):

	A	B	C	D
A	∞	1	1	1
B	∞	∞	∞	1
C	∞	∞	∞	∞
D	∞	∞	1	∞



- Can store weights in cells
- Space: $\Theta(V^2)$
- Time:
  - To visit each node that is adjacent to node u: $\Theta(V)$
  - To determine if node u is adjacent to node v: $\Theta(1)$

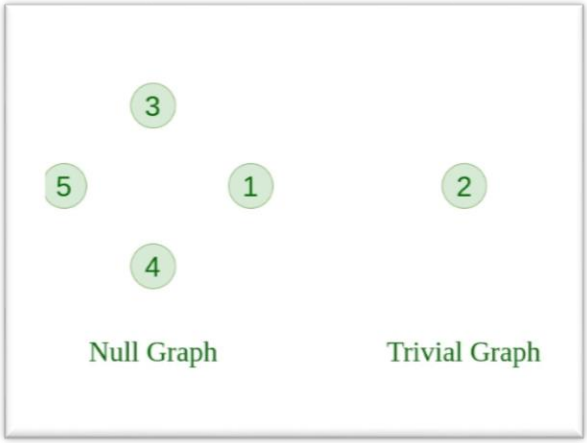
Types Of Graph

1. **Null Graph**

A graph is known as a null graph if there are no edges in the graph.

2. **Trivial Graph**

Graph having only a single vertex, it is also the smallest graph possible.

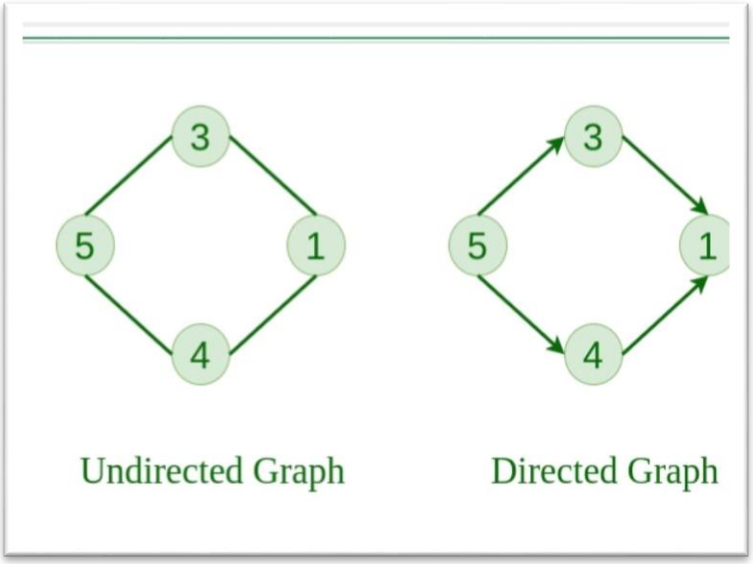


3. **Undirected Graph**

A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.

4. **Directed Graph**

A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.

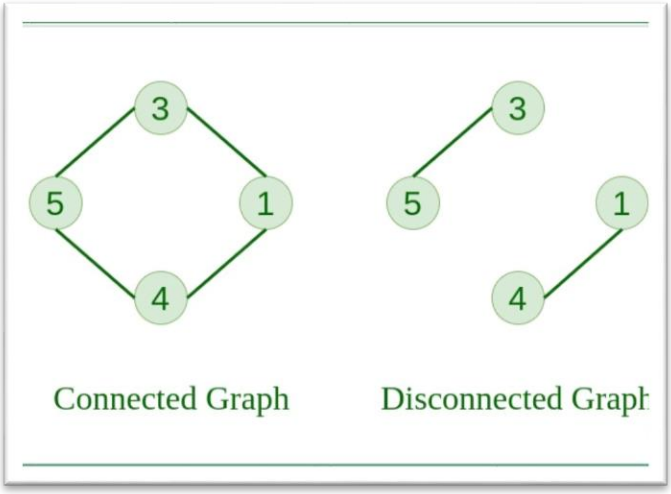


5**. Connected Graph**

The graph in which from one node we can visit any other node in the graph is known as a connected graph.

6. **Disconnected Graph**

The graph in which at least one node is not reachable from a node is known as a disconnected graph.

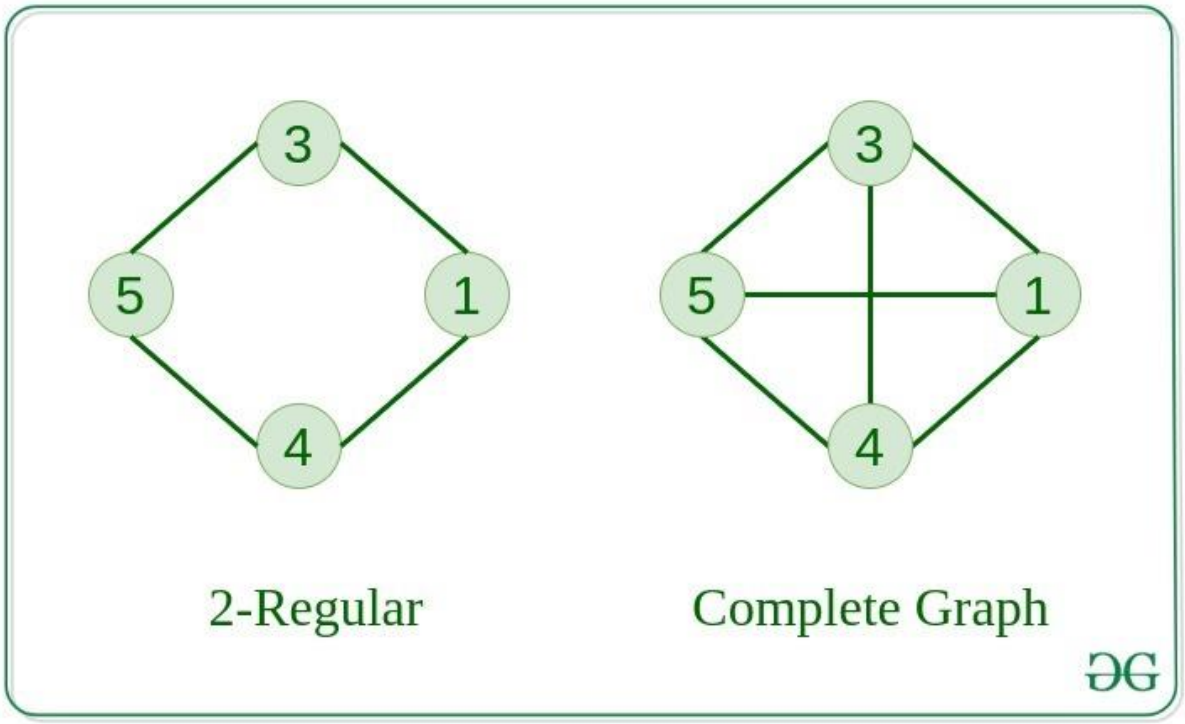


7. **Regular Graph**

The graph in which the degree of every vertex is equal to K is called K regular graph.

8. **Complete Graph**

The graph in which from each node there is an edge to each other node.



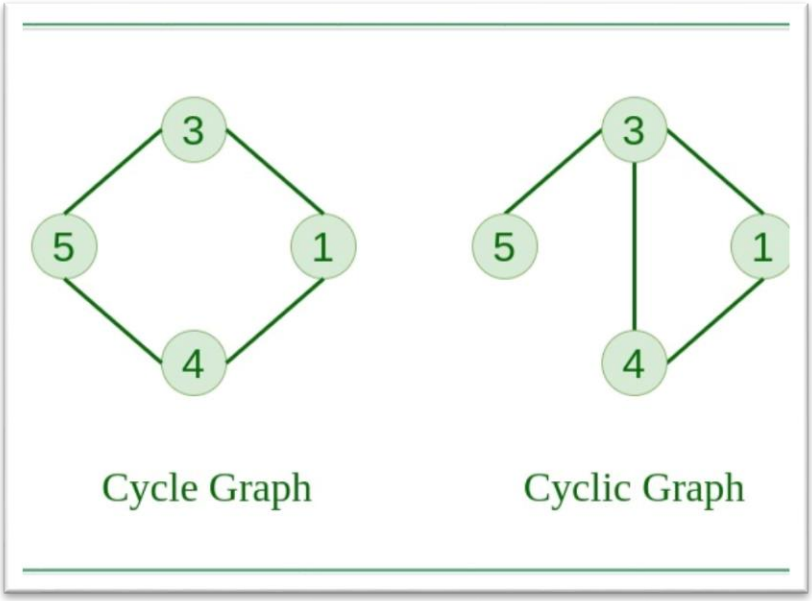
.

9. **Cycle Graph**

The graph in which the graph is a cycle in itself, the degree of each vertex is 2.

10. **Cyclic Graph**

A graph containing at least one cycle is known as a Cyclic graph.

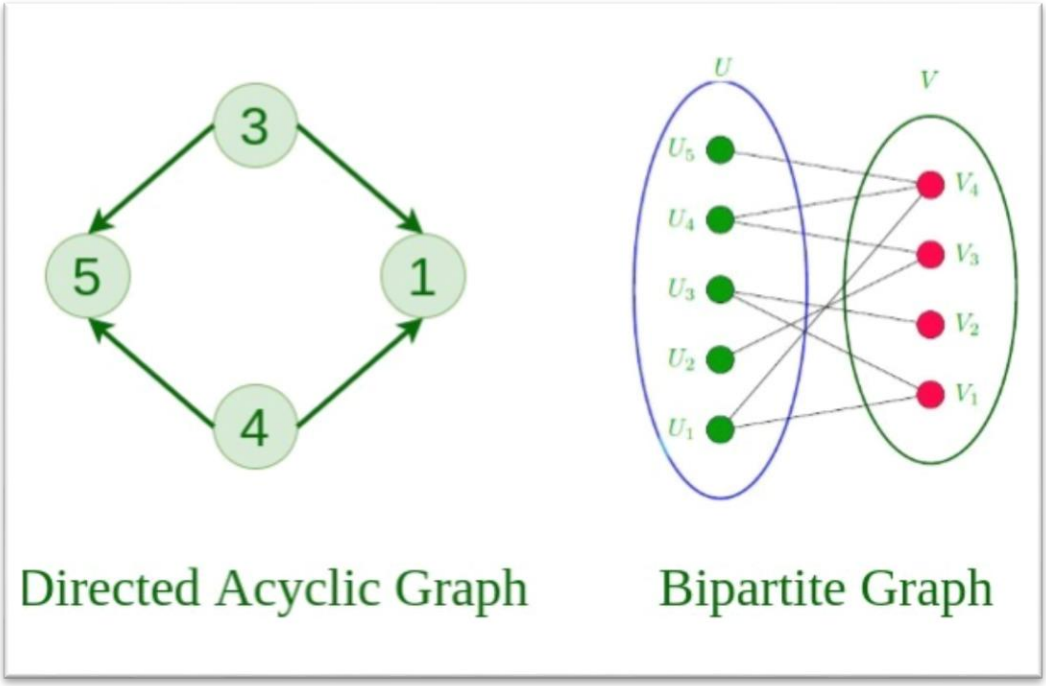


11. **Directed Acyclic Graph**

A Directed Graph that does not contain any cycle.

12. **Bipartite Graph**

A graph in which vertex can be divided into two sets such that vertex in each set does not contain any edge between them.

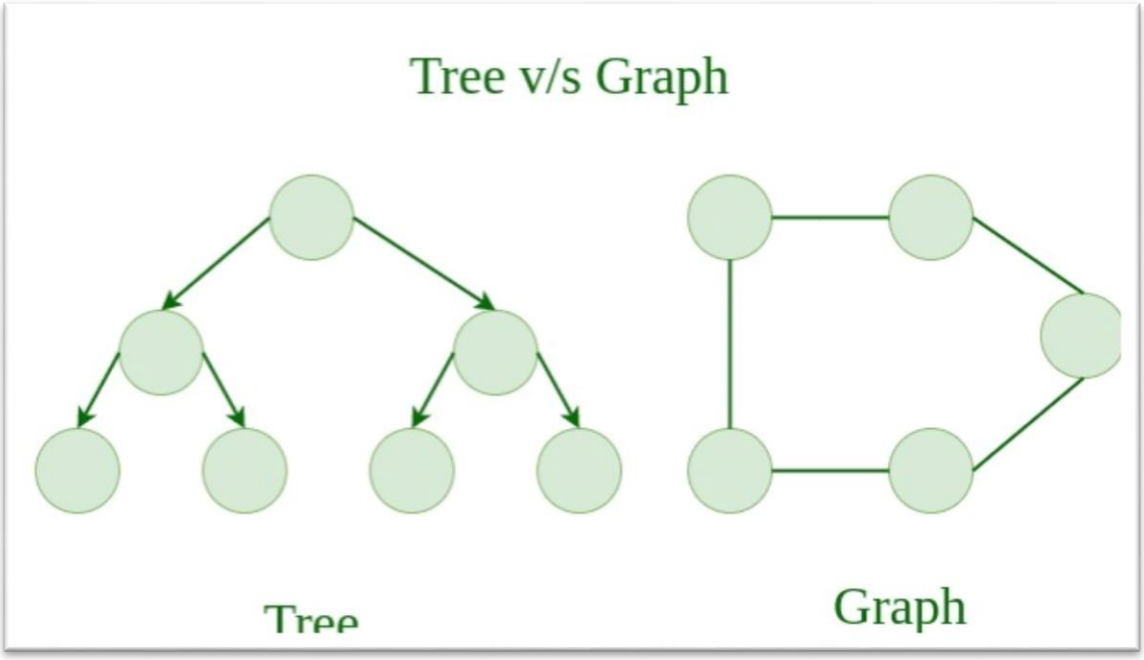


13. Weighted Graph

- A graph in which the edges are already specified with suitable weight is known as a weighted graph.
- Weighted graphs can be further classified as directed weighted graphs and undirected weighted graphs.

Tree v/s Graph

Trees are the restricted types of graphs, just with some more rules. Every tree will always be a graph but not all graphs will be trees. Linked List, Trees, and Heaps all are special cases of graphs.



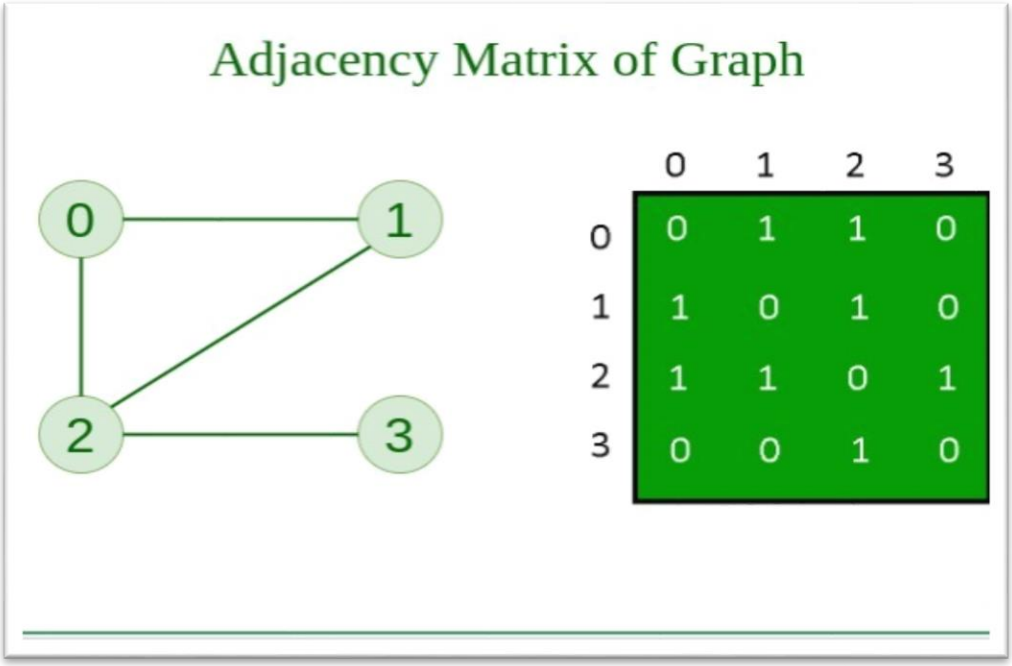
Representation of Graphs

There are two ways to store a graph:

- Adjacency Matrix
- Adjacency List

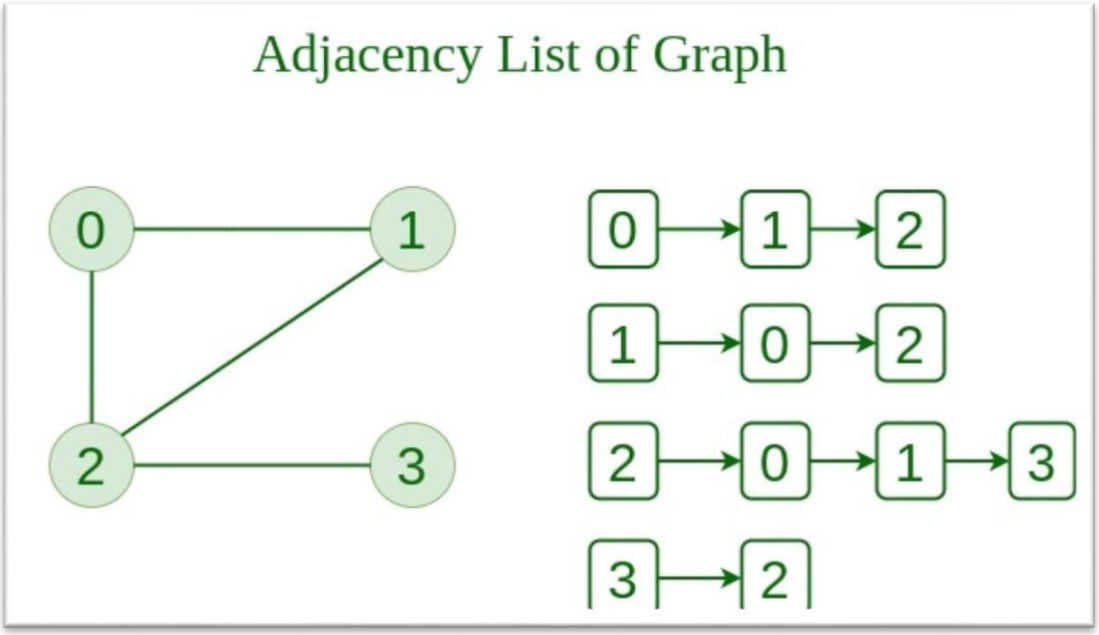
Adjacency Matrix

In this method, the graph is stored in the form of the 2D matrix where rows and columns denote vertices. Each entry in the matrix represents the weight of the edge between those vertices.



Adjacency List

This graph is represented as a collection of linked lists. There is an array of pointer which points to the edges connected to that vertex.



Comparison between Adjacency Matrix and Adjacency List

When the graph contains a large number of edges then it is good to store it as a matrix because only some entries in the matrix will be empty. An algorithm such as Prim's and Dijkstra adjacency matrix is used to have less complexity.

Action	Adjacency Matrix	Adjacency List
Adding Edge	O(1)	O(1)
Removing an edge	O(1)	O(N)
Initializing	O(N*N)	O(N)

Basic Operations on Graphs

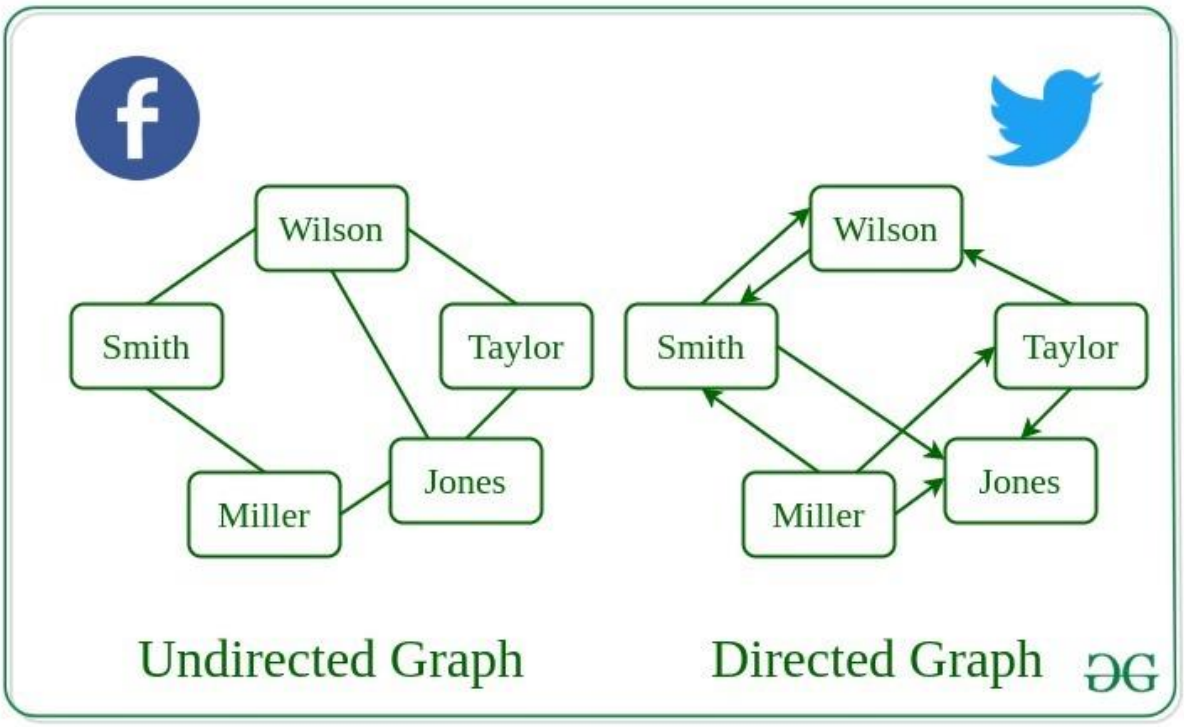
Below are the basic operations on the graph:

- Insertion of Nodes/Edges in the graph – Insert a node into the graph.
- Deletion of Nodes/Edges in the graph – Delete a node from the graph.
- Searching on Graphs – Search an entity in the graph.
- Traversal of Graphs – Traversing all the nodes in the graph.

Usage of graphs

- Maps can be represented using graphs and then can be used by computers to provide various services like the shortest path between two cities.
- When various tasks depend on each other then this situation can be represented using a Directed Acyclic graph and we can find the order in which tasks can be performed using topological sort.
- State Transition Diagram represents what can be the legal moves from current states. In-game of tic tac toe this can be used.

Real-Life Applications of Graph



Following are the real-life applications:

- Graph data structures can be used to represent the interactions between players on a team, such as passes, shots, and tackles. Analyzing these interactions can provide insights into team dynamics and areas for improvement.
- Commonly used to represent social networks, such as networks of friends on social media.
- Graphs can be used to represent the topology of computer networks, such as the connections between routers and switches.
- Graphs are used to represent the connections between different places in a transportation network, such as roads and airports.
- **Neural Networks:** Vertices represent neurons and edges represent the synapses between them. Neural networks are used to understand how our brain works and how connections change when we learn. The human brain has about 10^11 neurons and close to 10^15 synapses.

- **Compilers:** Graphs are used extensively in compilers. They can be used for type inference, for so-called data flow analysis, register allocation, and many other purposes. They are also used in specialized compilers, such as query optimization in database languages.
- **Robot planning:** Vertices represent states the robot can be in and the edges the possible transitions between the states. Such graph plans are used, for example, in planning paths for autonomous vehicles.

When to use Graphs:

- When you need to represent and analyze the relationships between different objects or entities.
- When you need to perform network analysis.
- When you need to identify key players, influencers or bottlenecks in a system.
- When you need to make predictions or recommendations.

ELEMENTARY GRAPH OPERATIONS

Depth First Search or DFS for a Graph

Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree. The only catch here is, that, unlike trees, graphs may contain cycles (a node may be visited twice).

To avoid processing a node more than once, use a boolean visited array. A graph can have more than one DFS traversal.

Example:

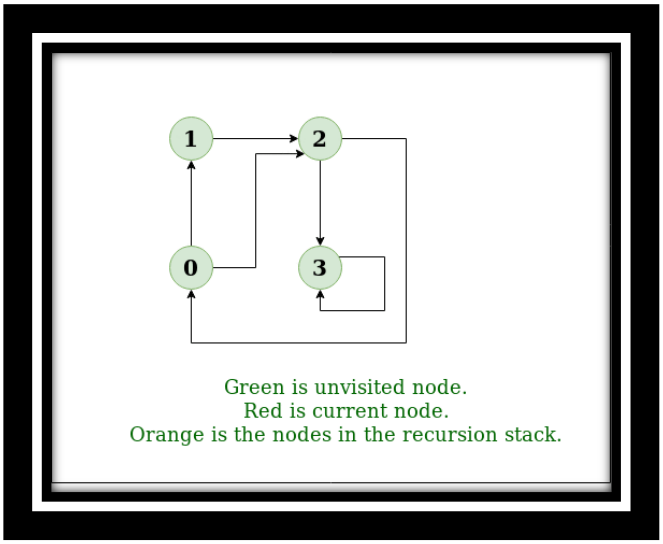
Input: n = 4, e = 6

0 -> 1, 0 -> 2, 1 -> 2, 2 -> 0, 2 -> 3, 3 -> 3

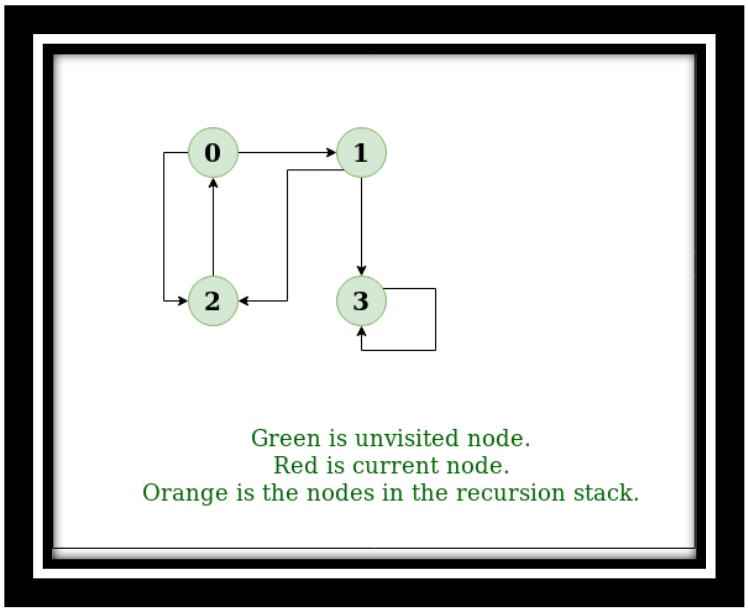
Output: DFS from vertex 1 : 1 2 0 3

Explanation:

DFS Diagram:



Input: n = 4, e = 6
2 -> 0, 0 -> 2, 1 -> 2, 0 -> 1, 3 -> 3, 1 -> 3
Output: DFS from vertex 2 : 2 0 1 3
Explanation:
DFS Diagram:



DFS of Graph

Depth-first search is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking. So the basic idea is to start from the root or any arbitrary node and mark the node and move to the adjacent unmarked node and continue this loop until there is no unmarked adjacent node. Then backtrack and check for other unmarked nodes and traverse them. Finally, print the nodes in the path.

Follow the below steps to solve the problem:

- Create a recursive function that takes the index of the node and a visited array.
- Mark the current node as visited and print the node.
- Traverse all the adjacent and unmarked nodes and call the recursive function with the index of the adjacent node.

The breadth-first search (BFS) algorithm is used to search a tree or graph data structure for a node that meets a set of criteria. It starts at the tree's root or graph and searches/visits all nodes at the current depth level before moving on to the nodes at the next depth level. Breadth-first search can be used to solve many problems in graph theory.

Breadth-First Traversal (or Search) for a graph is similar to the Breadth-First Traversal of a tree

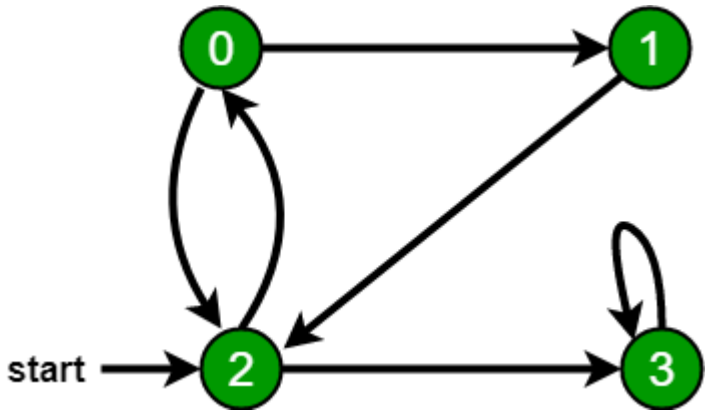
The only catch here is, that, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we divide the vertices into two categories:

- Visited and
- Not visited.

A boolean visited array is used to mark the visited vertices. For simplicity, it is assumed that all vertices are reachable from the starting vertex. BFS uses a **queue data structure** for traversal.

Example:

In the following graph, we start traversal from vertex 2.



When we come to **vertex 0**, we look for all adjacent vertices of it.

- 2 is also an adjacent vertex of 0.
- If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process.

There can be multiple BFS traversals for a graph. Different BFS traversals for the above graph :

2, 3, 0, 1

2, 0, 3, 1

BFS of graph

Implementation of BFS traversal on Graph:

Pseudocode:

Breadth_First_Search(Graph, X) // Here, Graph is the graph that we already have and X is the source node

Let Q be the queue

Q.enqueue(X) // Inserting source node X into the queue

Mark X node as visited.

While (Q is not empty)

Y = Q.dequeue( ) // Removing the front node from the queue

Process all the neighbors of Y, For all the neighbors Z of Y

If Z is not visited, Q. enqueue(Z) // Stores Z in Q

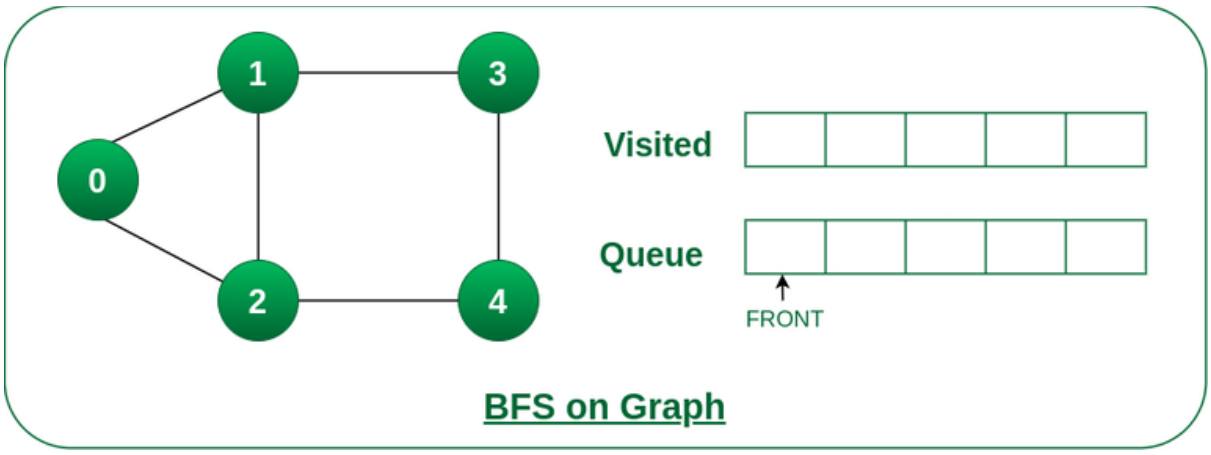
Mark Z as visited

Follow the below method to implement BFS traversal.

- Declare a queue and insert the starting vertex.
- Initialize a **visited** array and mark the starting vertex as visited.
- Follow the below process till the queue becomes empty:
    - Remove the first vertex of the queue.
    - Mark that vertex as visited.
    - Insert all the unvisited neighbors of the vertex into the queue.

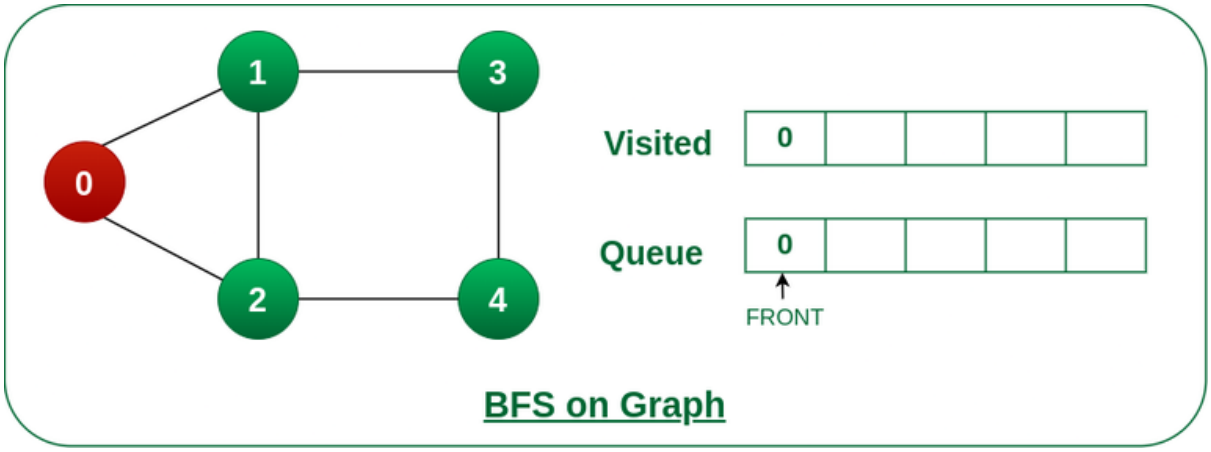
Illustration:

Step1: Initially queue and visited arrays are empty.



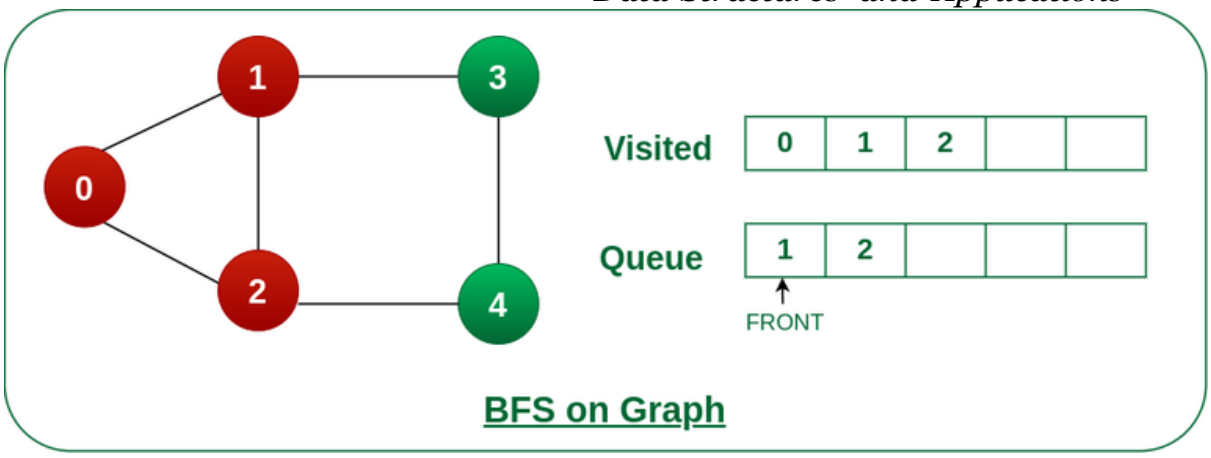
Queue and visited arrays are empty initially.

Step2: Push node 0 into queue and mark it visited.



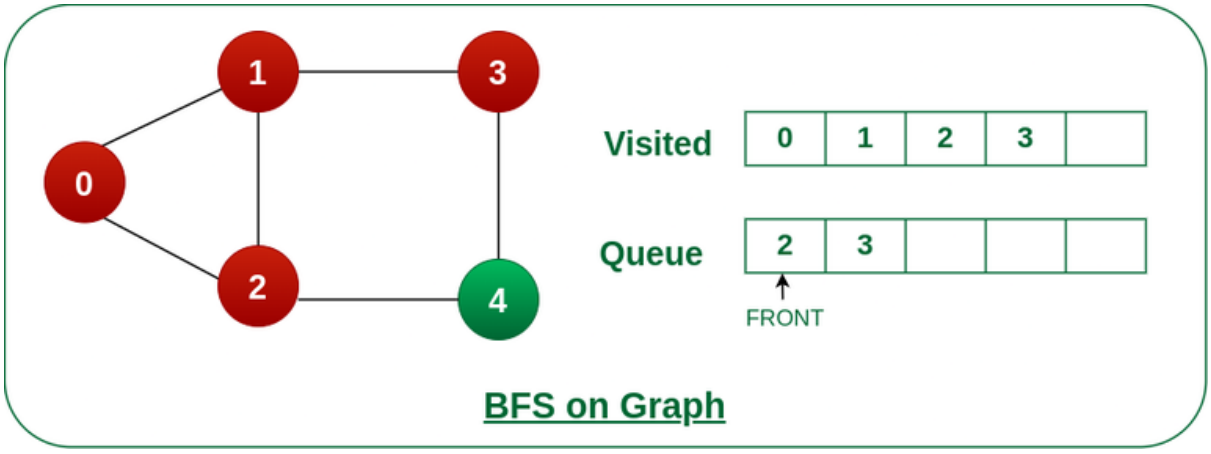
Push node 0 into queue and mark it visited.

Step 3: Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.



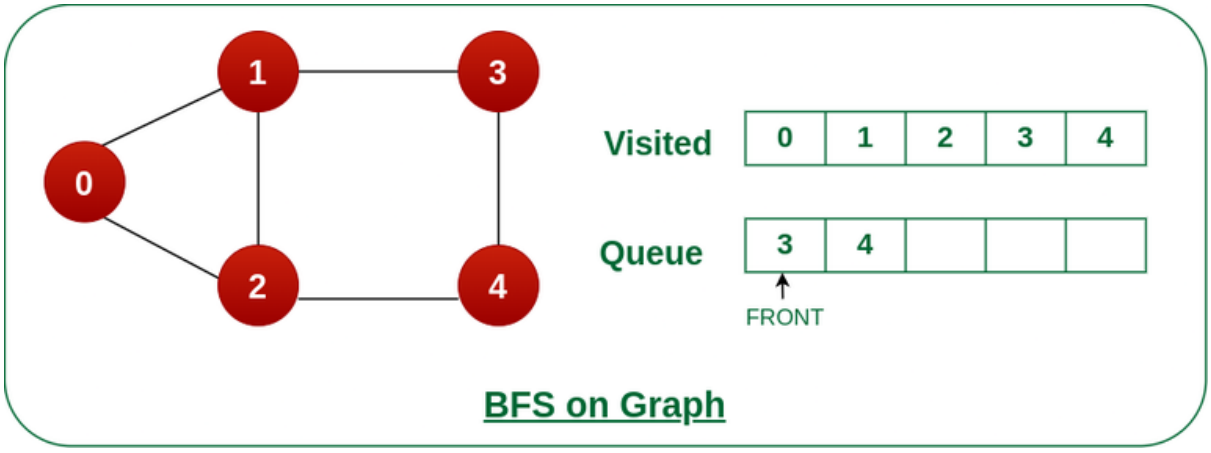
Remove node 0 from the front of queue and visited the unvisited neighbours and push into queue.

Step 4: Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



Remove node 1 from the front of queue and visited the unvisited neighbours and push

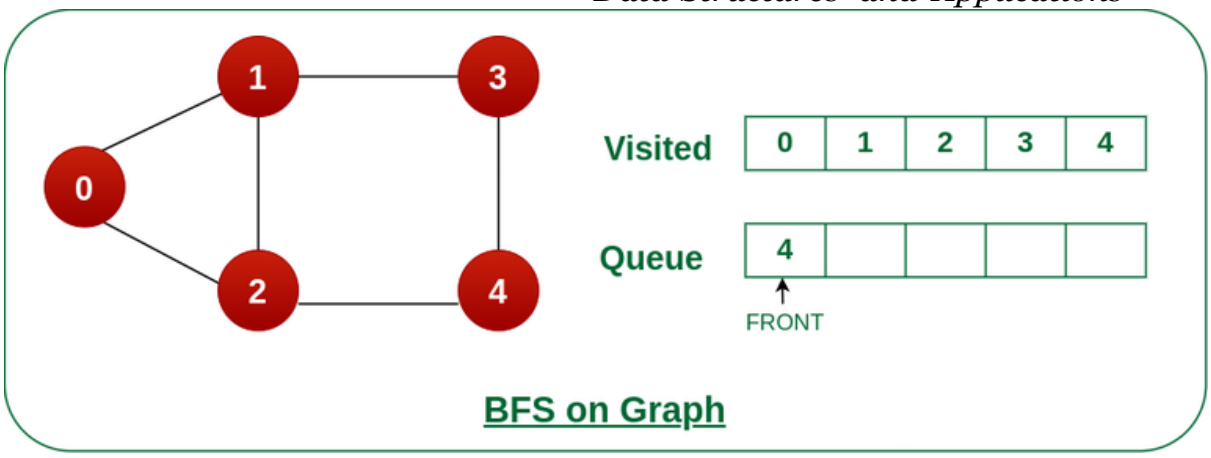
Step 5: Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.

Step 6: Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.

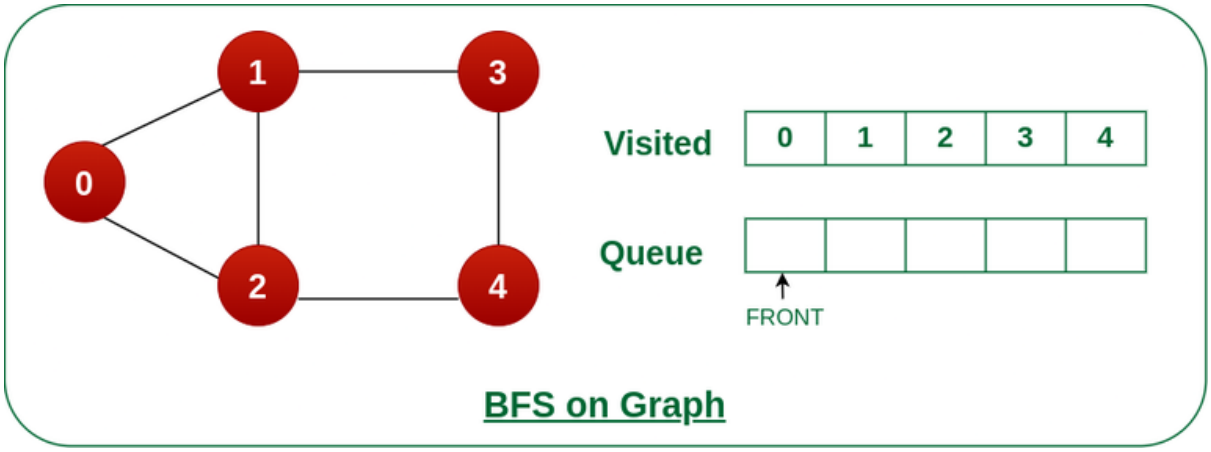
As we can see that every neighbours of node 3 is visited, so move to the next node that are in the front of the queue.



Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.

Steps 7: Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.

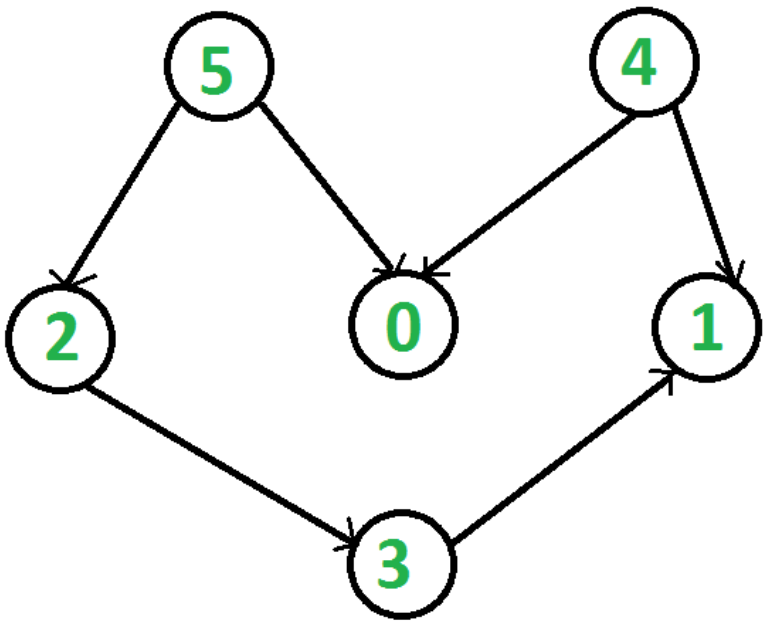
As we can see that every neighbours of node 4 are visited, so move to the next node that is in the front of the queue.



Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.

Now, Queue becomes empty, So, terminate these process of iteration.

Topological sorting for Directed Acyclic Graph (DAG) is a linear ordering of vertices such that for every directed edge u v, vertex u comes before v in the ordering.
Note: Topological Sorting for a graph is not possible if the graph is not a DAG.
For example, a topological sorting of the following graph is "5 4 2 3 1 0". There can be more than one topological sorting for a graph. Another topological sorting of the following graph is "4 5 2 3 1 0". The first vertex in topological sorting is always a vertex with an in-degree of 0 (a vertex with no incoming edges).



Topological sort

Topological Sorting vs Depth First Traversal (DFS)**:**

In DFS, we print a vertex and then recursively call DFS for its adjacent vertices. In topological sorting, we need to print a vertex before its adjacent vertices.
For example, in the given graph, the vertex '5' should be printed before vertex '0', but unlike DFS, the vertex '4' should also be printed before vertex '0'. So Topological sorting is different from DFS.
For example, a DFS of the shown graph is "5 2 3 1 0 4", but it is not a topological sorting.

Algorithm for Topological Sorting:

Prerequisite: DFS

We can modify DFS to find the Topological Sorting of a graph. In DFS,

- We start from a vertex, we first print it, and then
- Recursively call DFS for its adjacent vertices.

In topological sorting,

- We use a temporary stack.
- We don't print the vertex immediately,
- we first recursively call topological sorting for all its adjacent vertices, then push it to a stack.
- Finally, print the contents of the stack.

Note: A vertex is pushed to stack only when all of its adjacent vertices (and their adjacent vertices and so on) are already in the stack

Approach:

- Create a **stack** to store the nodes.
- Initialize **visited** array of size **N** to keep the record of visited nodes.
- Run a loop from **0** till **N**
    - if the node is not marked **True** in **visited** array
        - Call the recursive function for topological sort and perform the following steps.
            - Mark the current node as **True** in the **visited** array.
            - Run a loop on all the nodes which has a directed edge to the current node
                - if the node is not marked **True** in the **visited** array:
                - Recursively call the topological sort function on the node
            - Push the current node in the stack.
- Print all the elements in the stack.

Below image is an illustration of the above approach: